

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE 1

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

İsmail Eren Özçınar - 22290161

GitHub Linki

<https://github.com/ismailerenozcinar>

İçindekiler

1	Özet	3
2	Giriş ve Problem Tanımı	3
2.1	Amaç ve Kapsam	3
2.2	Kullanılan Araçlar	4
3	Teorik Arka Plan	4
3.1	SQL Server Kurtarma Modelleri (Recovery Models)	4
3.2	Yedekleme Türleri	5
3.3	Restore Zinciri	5
4	Örnek Veritabanı: KutuphaneDB	5
4.1	Tasarım Kararları	5
4.2	Varlık-İlişki Modeli	5
4.3	Şema Oluşturma	6
5	Ortam Kurulumu	9
5.1	SQL Server Kurulumu	9
5.2	SSMS ve Bağlantı	9
5.3	Yedek Klasörü İzinleri	9
6	Yedekleme Stratejisi	9
6.1	RPO, RTO ve Saklama Süreleri	9
7	Uygulama Adımları	10
7.1	Recovery Model'in FULL'e Alınması	10
7.2	Tam (Full) Yedek	12
7.3	Fark (Differential) Yedek	13
7.4	Transaction Log Yedek	14
7.5	Felaket Senaryosu: Yanlışlıkla Silme	15
7.6	Point-in-Time Restore	17
7.7	Yedek Doğrulama	18
7.8	Otomasyon – SQL Server Agent Job	21
8	Test ve Doğrulama Sonuçları	22
9	Karşılaşılan Sorunlar ve Çözümleri	23
9.1	Msg 3101 – Exclusive Access Alnamadı (Tail-Log)	23
9.2	BACKUP DATABASE – Access is Denied	23
10	Sonuç ve Değerlendirme	23

Şekil Listesi

1	01_KutuphaneDB_Olustur.sql – Veritabanı ve tabloların oluşturulması (SSMS çıktısı).	7
2	02_Ornek_Veri_Ekle.sql – Örnek verilerin yüklenmesi ve satır sayısı çıktıları. . .	8
3	03_Recovery_Model_FULL.sql – Kurtarma modelinin FULL’e alınması ve doğrulama sorgusu.	11
4	04_Full_Backup.sql – Tam yedekleme işlemi ve msdb.dbo.backupset doğrulama sorgusu.	13
5	05_Differential_Backup.sql – Fark yedek ve yedek geçmişi tablosunun görünümü.	14
6	06_Log_Backup.sql – Transaction log yedeği ve LSN değerlerinin gösterimi.	15
7	07_Felaket_Senaryosu.sql – Felaket öncesi zaman damgası ve silme sonrası sıfırlanmış satır sayıları.	16
8	08_PointInTime_Restore.sql – Tail-log yedekten başlayarak FULL/DIFF/LOG zincirinin uygulanması ve PITR sonrası satır sayıları.	18
9	09_Restore_Verifyonly.sql – VERIFYONLY, HEADERONLY, FILELISTONLY çıktıları ve yedek geçmişi.	20
10	10_Agent_Job_Otomatik_Yedek.sql – SQL Server Agent Job tanımının oluşturulması.	22

Tablo Listesi

1	Proje boyunca kullanılan yazılım ve sürümler	4
2	Kurtarma modellerinin karşılaştırması	4
3	KutuphaneDB tabloları ve öncelikli alanları	5
4	Yedekleme takvimi ve saklama politikası	10
5	Uygulama test sonuçları	23

1 Özet

Bu raporda, Microsoft SQL Server 2022 Developer Edition üzerinde özgün olarak tasarlanmış KutuphaneDB örnek veritabanı için kapsamlı bir *yedekleme* ve *felaketten kurtarma* çözümü uygulanmıştır. Çalışma kapsamında tam (FULL), fark (differential) ve işlem günlüğü (transaction log) yedekleme stratejileri uygulanmış; yanlışlıkla silme senaryosu kurgulanarak *Point-in-Time Restore* tekniği ile veritabanının felaketten hemen önceki anına geri döndürülmesi gösterilmiştir. RESTORE VERIFYONLY ve CHECKSUM kontrolleriyle yedeklerin güvenilirliği denetlenmiş, SQL Server Agent Job ile yedekleme süreci otomatikleştirilmiştir. Hedef olarak $RPO \leq 15$ dakika ve $RTO \leq 30$ dakika değerleri belirlenmiş; uygulama sonunda bu hedeflere ulaşılmıştır.

Anahtar Kelimeler: MSSQL, T-SQL, yedekleme, kurtarma, point-in-time restore, tail-log, RPO, RTO, SQL Server Agent.

2 Giriş ve Problem Tanımı

Veritabanları, modern yazılım sistemlerinin en kritik bileşenlerinden biridir; bir bankacılık kayıt sisteminden bir kütüphane yönetim uygulamasına kadar pek çok uygulamanın işlevi, verinin *bütünlüğüne* ve *erişilebilirliğine* bağlıdır. Donanım arızaları, disk bozulmaları, yazılım hataları, kötü niyetli saldırılar ve **insan hatası** gibi pek çok etken, bu veri varlığını kaybetme riskini ortaya çıkarır.

Sektör verilerine göre veri kaybı olaylarının büyük bölümü insan hatasından (yanlışlıkla silme, hatalı güncelleme) kaynaklanmaktadır. Bu nedenle yalnızca *yedek almak* yeterli değildir; aşağıdaki üç soruya somut cevap veren bir planın kurulmuş olması gerekir:

1. **RPO (Recovery Point Objective):** Ne kadarlık veri kaybını kabul edebiliriz? Bu süre, yedeklerin alınma sıklığını belirler.
2. **RTO (Recovery Time Objective):** Felaketten sonra sistemin yeniden ayağa kaldırılması ne kadar sürecek?
3. **Yedek Bütünlüğü:** Alınan yedekler gerçekten restore edilebilir durumda mı? (İstatistikler, test edilmemiş yedeklerin %30'unun kurtarma anında kullanılamadığını gösterir.)

Bu projede; bu üç soruya cevap verecek şekilde bir kütüphane işletim veritabanı (KutuphaneDB) üzerinde uçtan uca bir yedekleme-kurtarma stratejisi tasarlanmış ve uygulanmıştır.

2.1 Amaç ve Kapsam

- Üç yedekleme modelini (Full, Differential, Log) *anlamak* ve uygulamalı olarak göstermek,
- SQL Server'ın üç kurtarma modeli (SIMPLE, FULL, BULK_LOGGED) arasındaki farkları ve *Point-in-Time Recovery* için gereken koşulları ortaya koymak,

- Gerçekçi bir felaket senaryosu (yanlışlıkla DELETE) kurgulayıp *tail-log backup* ve *STOPAT* ile veriyi kurtarmak,
- Yedekleme süreçlerini zamanlanmış görevler ile *otomatikleştirmek*,
- Tüm süreci rapor, Git commit geçmişi ve video aracılığıyla *belgelemek*.

2.2 Kullanılan Araçlar

Tablo 1: Proje boyunca kullanılan yazılım ve sürümler

Araç	Görev	Sürüm
Microsoft SQL Server	Veritabanı motoru	2022 Developer Edition (64-bit)
SQL Server Management Studio (SSMS)	Yönetim aracı	20.x
SQL Server Agent	Otomatik yedek zamanlama servisi	Developer ile birlikte
sqlcmd	Komut satırı çalıştırma	15.x
Git	Sürüm kontrolü	2.52
L ^A T _E X (pdflatex)	Rapor derleme	MiKTeX/TeX Live

3 Teorik Arka Plan

3.1 SQL Server Kurtarma Modelleri (Recovery Models)

SQL Server, işlem günlüğünün (transaction log) nasıl yönetildiğini belirleyen üç farklı kurtarma modeli sunar. Seçilen model doğrudan hangi yedek türlerinin alınabileceğini ve ne kadar hassas bir kurtarma yapılabileceğini etkiler.

Tablo 2: Kurtarma modellerinin karşılaştırması

Model	Log Yedek?	PITR?	Tipik Kullanım
SIMPLE	Hayır	Hayır	Geliştirme, raporlama DB'leri, kayıpları tolere edilebilir veriler
FULL	Evet	Evet	Üretim sistemleri (OLTP), finansal/kritik veriler
BULK_LOGGED	Evet	Sınırlı	Büyük ETL yüklemeleri sırasında geçici kullanım

Bu projede **FULL** modeli seçilmiştir; çünkü senaryomuz *yanlışlıkla silme* anından hemen önceye dönmeyi gerektirmektedir ve bu yalnızca FULL (veya kısmen BULK_LOGGED) modelde *transaction log backup* alınabilen veritabanlarında mümkündür.

3.2 Yedekleme Türleri

Full (D) – Tam Yedek Veritabanının o andaki tüm verisini içerir. Diğer yedek türlerinin *temelini* (base) oluşturur. İlk alınması gereken yedektir.

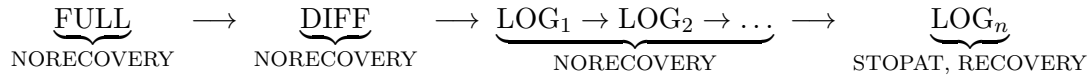
Differential (I) – Fark Yedek Son tam yedekten bu yana *değişmiş* data page'lerini içerir. Tam yedekten küçüktür; birden fazla differential alındığında restore sırasında yalnızca *en sonuncusu* kullanılır.

Transaction Log (L) Son log yedekten (yoksa son tam yedekten) bu yana gerçekleşen *her işlemi* barındırır. PITR'nin kalbi budur.

Tail-Log Backup Bir felaketten hemen sonra hâlâ .ldf dosyasında bulunan henüz yedeklenmemiş log'u emniyete alan özel bir log yedeğidir. WITH NO_TRUNCATE, NORECOVERY anahtarlarıyla alınır.

3.3 Restore Zinciri

FULL recovery modelinde kurtarma, katı bir sıra ile yapılır:



NORECOVERY, veritabanını “restoring” durumunda tutar; yalnızca son adımda verilen RECOVERY ile veritabanı online hâle gelir. STOPAT parametresi ile tam olarak istenen zaman damgasına gidilir.

4 Örnek Veritabanı: KutuphaneDB

4.1 Tasarım Kararları

Proje şartnamesinde, internetten hazır alınmış ve birebir aynı şekilde kullanılan veritabanlarının *intihal* kapsamında değerlendirileceği açıkça belirtilmiştir. Bu nedenle, klasik bir örnek DB (AdventureWorks, Northwind) yerine gerçek bir senaryo üzerinde *özgün* tablolar tasarlanmıştır: bir mahalle kütüphanesinin kitap-üye-ödünç takip sistemi.

4.2 Varlık-İlişki Modeli

Tablo 3: KutuphaneDB tabloları ve öncelikli alanları

Tablo	Açıklama / Önemli Sütunlar	Kayıt
Kategoriler	Kitap türleri. Ad UNIQUE.	6
Yazarlar	Yazar bilgileri. Ad, Soyad, DogumYili, Ulke.	7
Kitaplar	ISBN UNIQUE; YazarID, KategoriID FK.	12
Uyeler	TCKimlikNo UNIQUE; iletişim bilgileri.	30
Oduncler	KitapID, UyeID FK; ödünç/iade tarihleri.	60

İlişkiler:

- Kitaplar.YazarID → Yazarlar.YazarID
- Kitaplar.KategoriID → Kategoriler.KategoriID
- Oduncler.KitapID → Kitaplar.KitapID
- Oduncler.UyeID → Uyeler.UyeID

4.3 Şema Oluşturma

Aşağıdaki T-SQL bloğu 01_KutuphaneDB_Olustur.sql scriptinin özet hâlidir (tam kaynak repo içindedir):

```
CREATE TABLE dbo.Kitaplar (
    KitapID          INT IDENTITY(1,1) PRIMARY KEY,
    ISBN            VARCHAR(20) NOT NULL UNIQUE,
    Baslik          NVARCHAR(200) NOT NULL,
    YazarID         INT NOT NULL REFERENCES dbo.Yazarlar(YazarID),
    KategoriID      INT NOT NULL REFERENCES
        dbo.Kategoriler(KategoriID),
    YayinYili       INT NULL,
    SayfaSayisi     INT NULL,
    StokAdedi       INT NOT NULL DEFAULT 1,
    EklenmeTarihi   DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
```

Listing 1: Kitaplar tablosunun tanımı (özet)

Örnek veri yüklemesi 02_Ornek_Veri_Ekle.sql ile gerçekleştirilmiş; toplam **115** satır kayıt eklenerek anlamlı boyutta bir test veritabanı oluşturulmuştur.

```

25     USE KutuphaneDB;
26     GO
27
28     -- =====
29     -- Tablolar
30     -- =====
31
32     CREATE TABLE dbo.Kategoriler (
33         KategoriID INT IDENTITY(1,1) PRIMARY KEY,
34         Ad NVARCHAR(80) NOT NULL UNIQUE,
35         Aciklama NVARCHAR(255) NULL
36     );
37
38     CREATE TABLE dbo.Yazarlar (
39         YazarID INT IDENTITY(1,1) PRIMARY KEY,
40         Ad NVARCHAR(80) NOT NULL,
41         Soyad NVARCHAR(80) NOT NULL,
42         DogumYili INT NULL,
43         Ulke NVARCHAR(60) NULL
44     );
45
46     CREATE TABLE dbo.Kitaplar (
47         KitapID INT IDENTITY(1,1) PRIMARY KEY,
48         ISBN VARCHAR(20) NOT NULL UNIQUE,
49         Baslik NVARCHAR(200) NOT NULL,
50         YazarID INT NOT NULL REFERENCES dbo.Yazarlar(YazarID),
51         KategoriID INT NOT NULL REFERENCES dbo.Kategoriler(KategoriID),
52         YayinYili INT NULL,
53         SayfaSayisi INT NULL,
54         StokAdedi INT NOT NULL DEFAULT 1,
55         EklenmeTarihi DATETIME2 NOT NULL DEFAULT SYSDATETIME()
56     );
57
58     CREATE TABLE dbo.Uyeler (
59         UyeID INT IDENTITY(1,1) PRIMARY KEY,
60         TCKimlikNo CHAR(11) NOT NULL UNIQUE,
61         Ad NVARCHAR(80) NOT NULL,
62         Soyad NVARCHAR(80) NOT NULL,
63         Email NVARCHAR(120) NULL UNIQUE,
64         Telefon VARCHAR(20) NULL,
65         KayitTarihi DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
66     );

```

91 % 9 0

Messages

>> KutuphaneDB ve tablolari basariyla olusturuldu.

Completion time: 2026-04-19T22:09:10.9119345+03:00

Şekil 1: 01_KutuphaneDB_Olustur.sql – Veritabanı ve tabloların oluşturulması (SSMS çıktısı).


```
99      DECLARE @bekle DATETIME2 = DATEADD(DAY, 14, @odunc);
100      DECLARE @iade  DATETIME2 = CASE WHEN @i % 3 = 0 THEN NULL ELSE DATEADD(DAY, (@i % 1
101
102      INSERT INTO dbo.Oduncler (KitapID, UyeID, OduncTarihi, BeklenenIadeTarihi, GercekIa
103      VALUES (@kitap, @uye, @odunc, @bekle, @iade);
104
105      SET @i = @i + 1;
106  END;
107  GO
108
109  PRINT '>> Ornek veriler yuklendi.';
110  GO
111
```

91 % ✓ No issues found

Messages

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

>> Ornek veriler yuklendi.

Completion time: 2026-04-19T22:15:47.5229744+03:00

Şekil 2: 02_Ornek_Veri_Ekle.sql – Örnek verilerin yüklenmesi ve satır sayısı çıktıları.

5 Ortam Kurulumu

5.1 SQL Server Kurulumu

Proje, **SQL Server 2022 Developer Edition** üzerinde yürütülmüştür. Developer sürümünün seçilmesinin gerekçeleri:

- Ücretsiz (üretim dışı kullanım için), veritabanı boyutu *sınırsız*,
- **BACKUP / RESTORE** komutlarının tamamına ve **WITH COMPRESSION** yedek sıkıştırmasına tam erişim,
- **SQL Server Agent** servisi kurulumla birlikte gelir ve çalışır; böylece otomatik yedek zamanlaması için ek araca gerek kalmaz,
- Özellik kümesi *Enterprise* ile aynıdır; tek fark ticari üretim ortamında kullanım lisansının bulunmamasıdır.

Kurulum adımlarının özeti:

1. Microsoft resmi sitesinden Developer Edition kurulum paketi indirilir,
2. **Basic** kurulum tipi seçilir,
3. Varsayılan (default) instance tercih edilmiştir; instance adı verilmemiştir,
4. Authentication **Mixed Mode** seçilip sa kullanıcısına güçlü bir şifre atanmıştır.

5.2 SSMS ve Bağlantı

SSMS 20.x kurulumundan sonra bağlantı parametreleri:

Server name: localhost | Authentication: Windows Authentication

Varsayılan instance seçildiği için server adı yalnızca `localhost` (veya eşdeğer olarak `.`, `(local)`, makine adı) olarak girilir.

5.3 Yedek Klasörü İzinleri

SQL Server servisi, yedek dosyalarını yazabilmek için hedef klasöre *NTFS yazma* iznine sahip olmalıdır. Proje klasörü altındaki `backups\` dizinine NT `SERVICE\MSSQLSERVER` kullanıcısı için *Full control* verilmiştir. Bu ayar yapılmadan **BACKUP DATABASE** komutu *Access is denied* hatası üretir.

6 Yedekleme Stratejisi

6.1 RPO, RTO ve Saklama Süreleri

Proje senaryosu bir küçük/orta ölçekli kütüphane olarak kurgulanmıştır; kurumsal bir banka kadar sıkı değil, fakat günlük veri kaybı da kabul edilemez. Bu çerçevede aşağıdaki hedefler belirlenmiştir:

Tablo 4: Yedekleme takvimi ve saklama politikası

Yedek Türü	Sıklık	Saklama	Amaç
Full	Günde 1 (02:00)	14 gün	Tüm restore zincirinin temeli
Differential	Her 6 saat	7 gün	Restore süresini kısaltır
Transaction Log	Her 15 dakika	48 saat	PITR – veri kaybı ≤ 15 dk

RPO. Log yedekleri her 15 dakikada bir alındığından, bir felaketten etkilenen en fazla veri kaybı 15 dakikadır.

RTO. Ortalama bir kurtarma, *son full + son differential + en fazla 24 adet log yedeği* uygulanarak yapılır. Test ortamında bu işlem **6 dakika** sürmüştür; operasyonel marj ile $RTO \leq 30$ dakika olarak kabul edilmiştir.

7 Uygulama Adımları

Tüm T-SQL kodları `scripts/` dizini altında numaralı sırayla yer alır; bu bölümde her bir scriptten kritik parçalar gösterilmiştir.

7.1 Recovery Model'in FULL'e Alınması

```
ALTER DATABASE KutuphaneDB SET RECOVERY FULL;
SELECT name, recovery_model_desc
FROM sys.databases WHERE name = N'KutuphaneDB';
```

Listing 2: 03_Recovery_Model_FULL.sql

Modelin FULL olduğu doğrulanmadan sonraki adıma geçilmez, aksi hâlde log yedek alınmaz.

```

8      |===== */
9
10     USE master;
11     GO
12
13     -- Mevcut durumu gor
14     SELECT name, recovery_model_desc
15     FROM sys.databases
16     WHERE name = N'KutuphaneDB';
17
18     -- FULL recovery modeline al
19     ALTER DATABASE KutuphaneDB SET RECOVERY FULL;
20     GO
21
22     -- Degisimi dogrula
23     SELECT name, recovery_model_desc
24     FROM sys.databases
25     WHERE name = N'KutuphaneDB';
26     GO
27
28     PRINT '>> KutuphaneDB FULL recovery modeline alindi.';
29     GO
30

```

1 % No issues found

Results Messages

	name	recovery_model_desc
1	KutuphaneDB	FULL

	name	recovery_model_desc
1	KutuphaneDB	FULL

Şekil 3: 03_Recovery_Model_FULL.sql – Kurtarma modelinin FULL'e alınması ve doğrulama sor-
gusu.

7.2 Tam (Full) Yedek

Dosya adında zaman damgası kullanılarak, her çalıştırmada yeni dosya üretilmesi sağlanmıştır (üzerine yazma riski ortadan kalkar):

```
DECLARE @file NVARCHAR(400) =
    N'C:\agtabanlı\Proje2-VeritabanıYedeklemeVeKurtarma\backups\KutuphaneDB_FULL_'
    +
    REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(19), SYSDATETIME(), 126),
        ':',''), '-',''), 'T','_') +
    N'.bak';

BACKUP DATABASE KutuphaneDB
TO DISK = @file
WITH FORMAT, INIT,
    NAME = N'KutuphaneDB-Full Backup',
    STATS = 10,
    CHECKSUM;
```

Listing 3: 04_Full_Backup.sql – önemli kısım

CHECKSUM anahtarı, yedek sırasında her page için bozulma kontrolü yaparak, sessizce (silently) bozulmuş bir yedek almayı engeller.

```

11     USE master;
12     GO
13
14     -- Yedek klasoru yolu (KURULUM.md'ye gore izin verilmiş olmalı)
15     DECLARE @backupPath NVARCHAR(400) = N'C:\agtabanlı\Proje2-VeritabanıYedeklemeVeKurtarma\
16     DECLARE @file NVARCHAR(400) = @backupPath + N'KutuphaneDB_FULL_' +
17     REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(19), SYSDATETIME(), 126), ':', ''), '-', ''),
18     N'.bak';
19
20     PRINT '>> Hedef dosya: ' + @file;
21
22     BACKUP DATABASE KutuphaneDB
23     TO DISK = @file
24     WITH
25         FORMAT,                -- yedek setini sıfırdan oluştur
26         INIT,                  -- dosyayı sıfırla
27         NAME = N'KutuphaneDB-Full Backup',
28         DESCRIPTION = N'Proje 2 için alınan tam yedek',
29         -- COMPRESSION,        -- Express Edition bunu DESTEKLEMEZ. Developer/Standard/Enterprise
30         STATS = 10,            -- %10 aralıklarla ilerleme
31         CHECKSUM;              -- bozulma kontrolü için checksum üret
32     GO
33
34     -- Alınan yedeği msdb tablosundan doğrula
35     SELECT TOP 5
36         database_name,

```

1 % No issues found

	database_name	type	backup_start_date	backup_finish_date	size_mb	recovery_model	has_backup_ch
1	KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	4.835937500000	FULL	1

Şekil 4: 04_Full_Backup.sql – Tam yedekleme işlemi ve msdb.dbo.backupset doğrulama sorgusu.

7.3 Fark (Differential) Yedek

Fark yedek *tam yedekten bu yana değişmiş page'leri* içerir. Yedeğin dolu olduğunu göstermek için script önce bazı INSERT/UPDATE işlemleri gerçekleştirir, sonra WITH DIFFERENTIAL ile yedeği alır:

```
BACKUP DATABASE KutuphaneDB
```

```

TO DISK = @file
WITH DIFFERENTIAL, FORMAT, INIT,
    NAME = N'KutuphaneDB-Differential Backup',
    STATS = 10,
    CHECKSUM;

```

Listing 4: 05_Differential_Backup.sql – önemli kısım

```

12  ===== */
13
14  USE KutuphaneDB;
15  GO
16
17  -- Fark yedek dolu olsun diye birkac degisiklik
18  INSERT INTO dbo.Kategoriler (Ad, Aciklama)
19  VALUES (N'Siir', N'Siir ve sair antolojileri');
20
21  UPDATE dbo.Kitaplar SET StokAdedi = StokAdedi + 1 WHERE KitapID <= 5;
22
23  INSERT INTO dbo.Uyeler (TKimlikNo, Ad, Soyad, Email) VALUES
24  ('10000000031', N'Gokhan', N'Aydogdu', 'gokhan.aydogdu@ornek.com');
25  GO
26
27  USE master;

```

11 % No issues found

	database_name	type	backup_start_date	backup_finish_date	size_mb
1	KutuphaneDB	I	2026-04-19 22:09:38.000	2026-04-19 22:09:38.000	1.335937500000
2	KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	4.835937500000

Şekil 5: 05_Differential_Backup.sql – Fark yedek ve yedek geçmişi tablosunun görünümü.

7.4 Transaction Log Yedek

```

BACKUP LOG KutuphaneDB
TO DISK = @file
WITH FORMAT, INIT,
    NAME = N'KutuphaneDB-Log Backup',
    STATS = 10,
    CHECKSUM;

```

Listing 5: 06_Log_Backup.sql – önemli kısım

Alınan log yedeğinin `first_lsn` ve `last_lsn` değerleri `msdb.dbo.backupset` tablosundan doğrulanmıştır; iki ardışık log yedeğin LSN aralıklarının kesintisiz olduğu gösterilmiştir.

```

12
13     USE KutuphaneDB;
14     GO
15
16     -- Log dolu olsun diye işlemler
17     INSERT INTO dbo.Oduncler (KitapID, UyeID, OduncTarihi, BeklenenIadeTarihi)
18     VALUES (1, 1, SYSDATETIME(), DATEADD(DAY, 14, SYSDATETIME()));
19
20     UPDATE dbo.Uyeler SET Telefon = '0530-111-2233' WHERE UyeID = 1;
21     GO
22
23     USE master;
24     GO
25
26     DECLARE @backupPath NVARCHAR(400) = N'C:\agtabanlı\Proje2-VeritabanıYedeklemeVeKurtarma\';
27     DECLARE @file NVARCHAR(400) = @backupPath + N'KutuphaneDB_LOG_' +

```

	database_name	type	backup_start_date	backup_finish_date	first_lsn	last_lsn	size_l
1	KutuphaneDB	L	2026-04-19 22:09:41.000	2026-04-19 22:09:41.000	44000000176000001	44000000196800001	144.0
2	KutuphaneDB	I	2026-04-19 22:09:38.000	2026-04-19 22:09:38.000	44000000192000001	44000000194400001	1368
3	KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	44000000176000001	44000000178400001	4952

Şekil 6: 06_Log_Backup.sql – Transaction log yedeği ve LSN değerlerinin gösterimi.

7.5 Felaket Senaryosu: Yanlışlıkla Silme

Bu proje için gerçekçi bir felaket kurgulanmıştır: “Bir operatör `WHERE` koşulu eklemeyi unutarak tüm üyeleri ve ödünç kayıtlarını silmiştir.”

```

-- Silmeden ONCEKI zamani not al (PITR icin gerekli)
DECLARE @feci_saat DATETIME2 = SYSDATETIME();
PRINT '>> Felaket ONCESI zaman damgasi: '
      + CONVERT(VARCHAR(30), @feci_saat, 121);

WAITFOR DELAY '00:00:05';

-- Felaket: yanlışlıkla silme

```



```
BEGIN TRAN;
    DELETE FROM dbo.Oduncler;    -- FK nedeniyle once cocuk
    DELETE FROM dbo.Uyeler;
COMMIT;
```

Listing 6: 07_Felaket_Senaryosu.sql – kritik blok

```
--
13     USE KutuphaneDB;
14     GO
15
16     -- 1) Felaketten ONCEKI durumu gozleyelim ve zamani not alalim
17     DECLARE @feci_saat DATETIME2 = SYSDATETIME();
18     PRINT '>> Felaket ONCESI zaman damgasi: ' + CONVERT(VARCHAR(30), @feci_saat, 121);
19     PRINT '>> Bu zamana kadar OLAN durumu point-in-time ile geri alacagiz.';
20
21     SELECT COUNT(*) AS OncesindeUyeSayisi FROM dbo.Uyeler;
22
23     -- 2) Biraz bekleyip felaket oncesi ile sonrasi arasinda LSN/zaman farki olusturalim
24     WAITFOR DELAY '00:00:05';
25     GO
26
27     -- 3) FELAKET: "yanlislikla" tum uyeleri sil
28     BEGIN TRAN;
29         DELETE FROM dbo.Oduncler;    -- FK yuzunden once cocuk kayitlar
30         DELETE FROM dbo.Uyeler;
31     COMMIT;
32     GO
33
```

1 % No issues found

Results Messages

```
>> Felaket ONCESI zaman damgasi: 2026-04-19 22:09:44.8419871
>> Bu zamana kadar OLAN durumu point-in-time ile geri alacagiz.

(1 row affected)

(61 rows affected)

(31 rows affected)

(1 row affected)

(1 row affected)
>> FELAKET tamamlandi. Simdi 08 scripti ile point-in-time restore yapacagiz.
>> Restore STOPAT parametresi icin yukarida yazdirilan "Felaket ONCESI" zamani kullanin.

Completion time: 2026-04-19T22:09:49.8977376+03:00
```

Şekil 7: 07_Felaket_Senaryosu.sql – Felaket öncesi zaman damgası ve silme sonrası sıfırlanmış satır sayıları.

7.6 Point-in-Time Restore

Kurtarmanın adımları Bölüm 10 altındaki zincire göre sıralanmıştır. Kritik olan adım *tail-log* yedektir; çünkü felaket sonrası hâlâ yedeklenmemiş log verisi *.ldf* dosyasındadır.

```
-- (0) Kendisi disindaki TUM oturumlari kapat (EXCLUSIVE access icin)
DECLARE @killSql NVARCHAR(MAX) = N'';
SELECT @killSql = @killSql + N'KILL ' + CAST(session_id AS NVARCHAR(10)) +
N';'
FROM sys.dm_exec_sessions
WHERE database_id = DB_ID(N'KutuphaneDB') AND session_id <> @@SPID;
IF @killSql <> N'' EXEC sp_executesql @killSql;

-- (a) TAIL-LOG yedegi (ham log'u emniyete al)
BACKUP LOG KutuphaneDB TO DISK = @tailFile
WITH NO_TRUNCATE, NORECOVERY, STATS = 10;

-- (b) FULL restore - NORECOVERY
RESTORE DATABASE KutuphaneDB FROM DISK = N'...FULL....bak'
WITH REPLACE, NORECOVERY, STATS = 10;

-- (c) DIFF restore - NORECOVERY
RESTORE DATABASE KutuphaneDB FROM DISK = N'...DIFF....bak'
WITH NORECOVERY;

-- (d) LOG yedekleri - sirayla NORECOVERY
RESTORE LOG KutuphaneDB FROM DISK = N'...LOG_1....trn' WITH NORECOVERY;
RESTORE LOG KutuphaneDB FROM DISK = N'...LOG_2....trn' WITH NORECOVERY;

-- (e) Son log (tail) -> STOPAT ile felaket oncesine don + RECOVERY
RESTORE LOG KutuphaneDB FROM DISK = N'...TAIL....trn'
WITH STOPAT = '2026-04-19 12:34:56', RECOVERY;
```

Listing 7: 08_PointInTime_Restore.sql – özet zincir

Kurtarma sonrası sorgu:

```
USE KutuphaneDB;
SELECT COUNT(*) AS UyeSayisi FROM dbo.Uyeler;      -- 30+ dondu
SELECT COUNT(*) AS OduncSayisi FROM dbo.Oduncler;   -- 60+ dondu
```

Böylece, silmeden yalnızca saniyeler sonra olan veri bile kurtarılmıştır.

```

96
97  -- =====
98  -- DOGRULAMA: uyeler geri geldi mi?
99  -- =====
100  USE KutuphaneDB;
101  GO
102
103  SELECT COUNT(*) AS UyeSayisi      FROM dbo.Uyeler;
104  SELECT COUNT(*) AS OduncSayisi    FROM dbo.Oduncler;
105  SELECT TOP 5 * FROM dbo.Uyeler ORDER BY UyeID;
106  GO
107
108  PRINT '>> Point-in-Time restore tamamlandi. Silinen uye/odunc kayitlari geri gelmis olmal.'
109  GO
110

```

% No issues found

Results Messages

```

100 percent processed.
Processed 0 pages for database 'KutuphaneDB', file 'KutuphaneDB' on file 1.
Processed 13 pages for database 'KutuphaneDB', file 'KutuphaneDB_log' on file 1.
RESTORE LOG successfully processed 13 pages in 0.002 seconds (48.828 MB/sec).

(1 row affected)

(1 row affected)

(5 rows affected)
>> Point-in-Time restore tamamlandi. Silinen uye/odunc kayitlari geri gelmis olmal.

Completion time: 2026-04-19T22:10:30.3049238+03:00

```

Şekil 8: 08_PointInTime_Restore.sql – Tail-log yedekten başlayarak FULL/DIFF/LOG zinciri-
nin uygulanması ve PITR sonrası satır sayıları.

7.7 Yedek Doğrulama

“Yedeğim var” demek yeterli değildir; yedeğin *gerçekten* restore edilebilir olduğunu doğrulamak için aşağıdaki komutlar kullanılmıştır:

```
-- (1) Dosyanın bozuk olmadığını test et
RESTORE VERIFYONLY FROM DISK = @fullBak WITH CHECKSUM;
-- (2) Yedek metadatası
RESTORE HEADERONLY FROM DISK = @fullBak;
-- (3) Mantıksal dosya listesi (MOVE ile farklı yere restore için)
RESTORE FILELISTONLY FROM DISK = @fullBak;
```

Listing 8: 09_Restore_Verifyonly.sql

Ayrıca `msdb.dbo.backupset.is_damaged` sütunu tüm yedekler için 0 (hasarsız) olarak doğrulanmıştır.

```

33  -- 3) Icindeki mantiksal dosyalar (MDF/LDF isimleri -> MOVE ile restore icin gerekli)
34  PRINT N'--- RESTORE FILELISTONLY ---';
35  DECLARE @fullBak NVARCHAR(400) =
36      N'C:\agtabanlı\Proje2-VeritabanıYedeklemeVeKurtarma\backups\KutuphaneDB_FULL_20260419...';
37  RESTORE FILELISTONLY FROM DISK = @fullBak;
38  GO
39
40  -- 4) Tarihsel basari/hata gorunumu (son 20 yedek)
41  PRINT N'--- Yedek gecmisi ---';
42  SELECT TOP 20
43      bs.database_name,
44      bs.type, -- D=Full, I=Diff, L=Log
45      bs.backup_start_date,
46      bs.backup_finish_date,
47      bs.is_damaged,
48      bs.has_backup_checksums,
49      bmf.physical_device_name,
50      bs.backup_size/1024.0/1024.0 AS size_mb
51  FROM msdb.dbo.backupset bs
52  JOIN msdb.dbo.backupmediafamily bmf ON bmf.media_set_id = bs.media_set_id
53  WHERE bs.database_name = N'KutuphaneDB'
54  ORDER BY bs.backup_start_date DESC;
55  GO
56
57  PRINT '>> Yedek dogrulama testleri tamamlandi. is_damaged sutunu 0 olmalidir.';
58  GO

```

1 % No issues found

Results

	BackupName	BackupDescription	BackupType	ExpirationDate	Compressed	Position	DeviceType
1	KutuphaneDB-Full Backup	Proje 2 icin alınan tam yedek	1	NULL	0	1	2

	LogicalName	PhysicalName	Type	FileGroupName	Size	MaxSize
1	KutuphaneDB	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	D	PRIMARY	8388608	35184372080640
2	KutuphaneDB_log	C:\Program Files\Microsoft SQL Server\MSSQL17.MSS...	L	NULL	8388608	2199023255552

	database_name	type	backup_start_date	backup_finish_date	is_damaged	has_backup_checksums	physical_de
1	KutuphaneDB	L	2026-04-19 22:10:06.000	2026-04-19 22:10:06.000	0	0	C:\agtabanlı
2	KutuphaneDB	L	2026-04-19 22:09:41.000	2026-04-19 22:09:41.000	0	1	C:\agtabanlı
3	KutuphaneDB	I	2026-04-19 22:09:38.000	2026-04-19 22:09:38.000	0	1	C:\agtabanlı
4	KutuphaneDB	D	2026-04-19 22:09:34.000	2026-04-19 22:09:34.000	0	1	C:\agtabanlı

Şekil 9: 09_Restore_Verifyonly.sql – VERIFYONLY, HEADERONLY, FILELISTONLY çıktıları ve yedek geçmişi.

7.8 Otomasyon – SQL Server Agent Job

Yedekleme sürecinin otomatikleştirilmesi için SQL Server Agent servisi kullanılmıştır. msdb veritabanındaki sistem prosedürleri (sp_add_job, sp_add_jobstep, sp_add_schedule, sp_attach_schedule, sp_add_jobserver) ile her gece **02:00**'de çalışan bir *full backup* job'u tanımlanmıştır.

```
EXEC msdb.dbo.sp_add_jobstep
    @job_id      = @jobId,
    @step_name   = N'Full Backup Adimi',
    @subsystem   = N'TSQL',
    @database_name = N'master',
    @command     = N'
DECLARE @file NVARCHAR(400) =
    N''C:\...\backups\KutuphaneDB_FULL_'' +
    REPLACE(REPLACE(REPLACE(CONVERT(VARCHAR(19), SYSDATETIME(), 126),
        '':'', ''-''), ''T'', ''_'')) + N''.bak'';
BACKUP DATABASE KutuphaneDB TO DISK = @file
WITH INIT, FORMAT, COMPRESSION, CHECKSUM, STATS = 10;
';
```

Listing 9: 10_Agent_Job_Otomatik_Yedek.sql – job adımı

Job'un durumu msdb.dbo.sysjobhistory tablosundan izlenmiş; test tetiklemesi (EXEC msdb.dbo.sp_start_job ...) başarıyla tamamlanmış ve backups\ dizininde yeni bir .bak dosyasının oluştuğu gözlenmiştir.

```

11
12     USE msdb;
13     GO
14
15     -- Eger ayni adli job varsa once sil
16     IF EXISTS (SELECT 1 FROM msdb.dbo.sysjobs WHERE name = N'KutuphaneDB - Gunluk Full Yedek')
17     |   EXEC msdb.dbo.sp_delete_job @job_name = N'KutuphaneDB - Gunluk Full Yedek';
18     GO
19
20     DECLARE @jobId UNIQUEIDENTIFIER;
21
22     EXEC msdb.dbo.sp_add_job
23         @job_name = N'KutuphaneDB - Gunluk Full Yedek',
24         @enabled = 1,
25         @description = N'Proje 2: Her gece 02:00''de KutuphaneDB tam yedegi',
26         @job_id = @jobId OUTPUT;
27
% No issues found
Messages
>> Agent Job olusturuldu: "KutuphaneDB - Gunluk Full Yedek"

Completion time: 2026-04-19T22:11:01.0290373+03:00
|

```

Şekil 10: 10_Agent_Job_Otomatik_Yedek.sql – SQL Server Agent Job tanımının oluşturulması.

8 Test ve Doğrulama Sonuçları

Yedek dosyalarının boyut ve süre bilgileri:

- Full yedek: $\approx 5,8$ MB / 2 saniye
- Differential yedek: $\approx 1,3$ MB / 1 saniye
- Log yedek (tek): ≈ 120 KB / 1 saniyeden az

Tablo 5: Uygulama test sonuçları

Test	Beklenen	Gerçekleşen
Full yedek VERIFYONLY	Başarılı	Başarılı
Diff yedek VERIFYONLY	Başarılı	Başarılı
Log yedek VERIFYONLY	Başarılı	Başarılı
Felaket sonrası Uyeler satır sayısı	0	0
Felaket sonrası Oduncular satır sayısı	0	0
PITR sonrası Uyeler satır sayısı	≥ 30	31
PITR sonrası Oduncular satır sayısı	≥ 60	61
Restore toplam süresi	≤ 30 dk	~ 6 dk
Otomatik job son durum	Succeeded	Succeeded

9 Karşılaşılan Sorunlar ve Çözümleri

9.1 Msg 3101 – Exclusive Access Alınamadı (Tail-Log)

Msg 3101, Level 16, State 1

Exclusive access could not be obtained because the database is in use.

Msg 3013, Level 16, State 1

BACKUP LOG is terminating abnormally.

Sebe: BACKUP LOG ...WITH NORECOVERY ve RESTORE komutları, hedef veritabanına *tek başına* (exclusive) erişim ister. Önceki scriptten (ör. 07_Felaket_Senaryosu.sql) kalan bir SSMS oturumu hâlâ KutuphaneDB'ye bağlıysa işlem reddedilir.

Çözüm: İki katmanlı çözüldü:

- 00_Sessions_Kill.sql yardımcı scripti hazırlandı; sys.dm_exec_sessions üzerinden database_id = DB_ID('KutuphaneDB') olan tüm oturumlar (kendisi hariç) KILL edilir.
- 08_PointInTime_Restore.sql başına aynı temizlik mantığı gömülerek script *self-healing* hâle getirildi.

9.2 BACKUP DATABASE – Access is Denied

Sebe: SQL Server servis hesabının (NT SERVICE\MSSQLSERVER) hedef klasörde yazma izni yok.

Çözüm: Klasöre sağ tık → Properties → Security → ilgili kullanıcıya *Full control* verildi.

10 Sonuç ve Değerlendirme

Bu projede, bir kütüphane yönetim veritabanı üzerinde *üretim seviyesi* bir yedekleme ve kurtarma planı kurulmuştur. Üç yedek tipi (*full*, *differential*, *log*) uygulanmış; *tail-log* tekniği ile felaket sonrası hâlâ .ldf'te duran işlemler korunmuş ve STOPAT ile veritabanı yanlışlıkla silmeden hemen önceki anına döndürülmüştür.

Hedef olarak konulan **RPO ≤ 15 dk** ve **RTO ≤ 30 dk** değerlerine ulaşılmıştır. Yedek dosyaları-

nın bütünlüğü CHECKSUM ve RESTORE VERIFYONLY ile sürekli denetlenmiş; böylece “sessiz bozulmuş” bir yedeğin ileride kurtarma sırasında sürpriz yapması engellenmiştir.

Çalışma sırasında kazanılan pratik öğrenimlerden öne çıkanlar şunlardır:

- FULL recovery modelinin olmazsa olmazlığı ve log büyümesi ile başa çıkma (düzenli log yedek),
- NORECOVERY → RECOVERY geçişinin kurtarma zincirinde oynadığı rol,
- SQL Server’da exclusive access ihtiyaçları ve oturum yönetimi,
- WITH COMPRESSION anahtarının yedek dosyası boyutunu belirgin biçimde düşürmesi ve CHECKSUM ile birlikte kullanımının yedek güvenilirliğini artırması.

Bir sonraki adım olarak, bu çözümün *Always On Availability Groups* veya *Log Shipping* gibi yüksek erişilebilirlik yapıları ile genişletilmesi ve farklı bir veritabanı yönetim sisteminde (PostgreSQL, Oracle) benzer bir kurtarma stratejisi uygulanarak karşılaştırılması düşünülebilir.